DSA Assignment #4: 线性结构

2500010774 兰玉琪 数学科学学院

1. 题目

E160.相交链表

hash table, linked list, two pinters, https://leetcode.cn/problems/intersection-of-two-linked-lists/

思路：

代码：

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, x):
#         self.val = x
#         self.next = None
class Solution:
    def getIntersectionNode(self, headA: ListNode, headB: ListNode) -> Optional[ListNod
        myset = set()
        while headA or headB:
            if headA:
                if headA not in myset:
                    myset.add(headA)
                    headA = headA.next
                else:
                    return headA
            if headB:
                if headB not in myset:
                    myset.add(headB)
                    headB = headB.next
                else:
                    return headB
        return None
```

	Status	Language	Runtime
1	Accepted Aug 25, 2025	Python3	76 ms

Python3 Auto

```python
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, x):
#         self.val = x
#         self.next = None
class Solution:
    def getIntersectionNode(self, headA: ListNode, headB: ListNode) -> Optional
```

Saved Ln 1, Col

Testcase | Test Result

You must run your code first

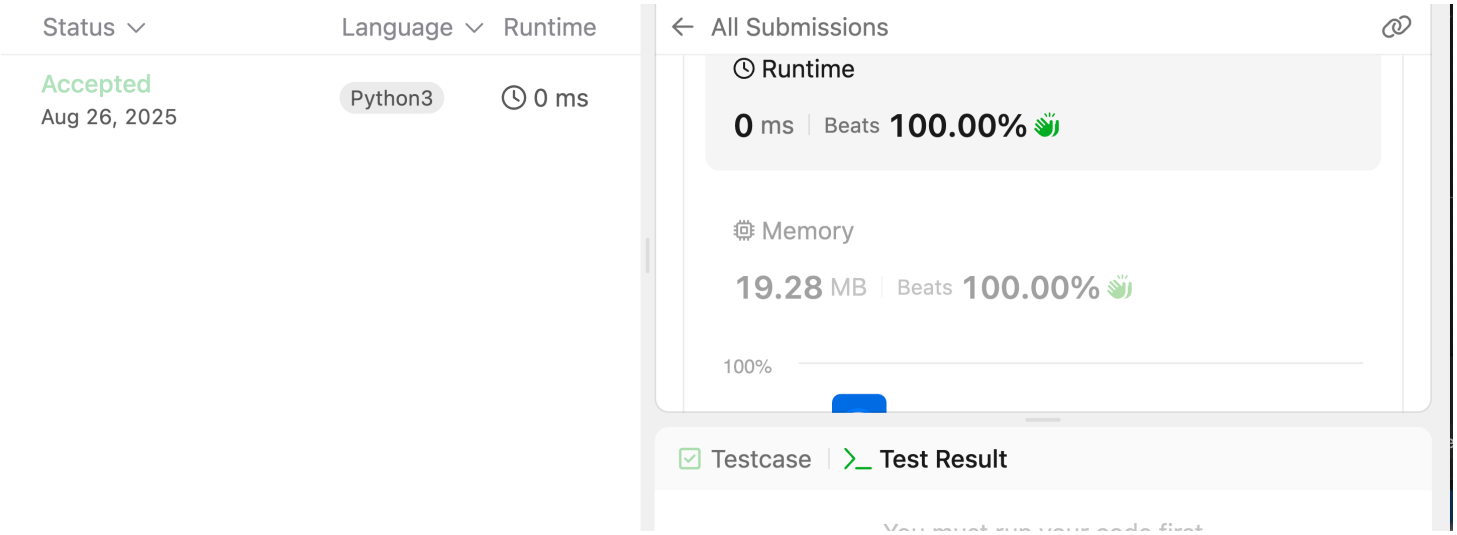
E206.反转链表

recursion, linked list, https://leetcode.cn/problems/reverse-linked-list/

思路:

代码:

```python
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        n1 = head
        n2 = None
        while n1:
            n2 = ListNode(n1.val,n2)
            n1 = n1.next
        return n2
```



M234.回文链表

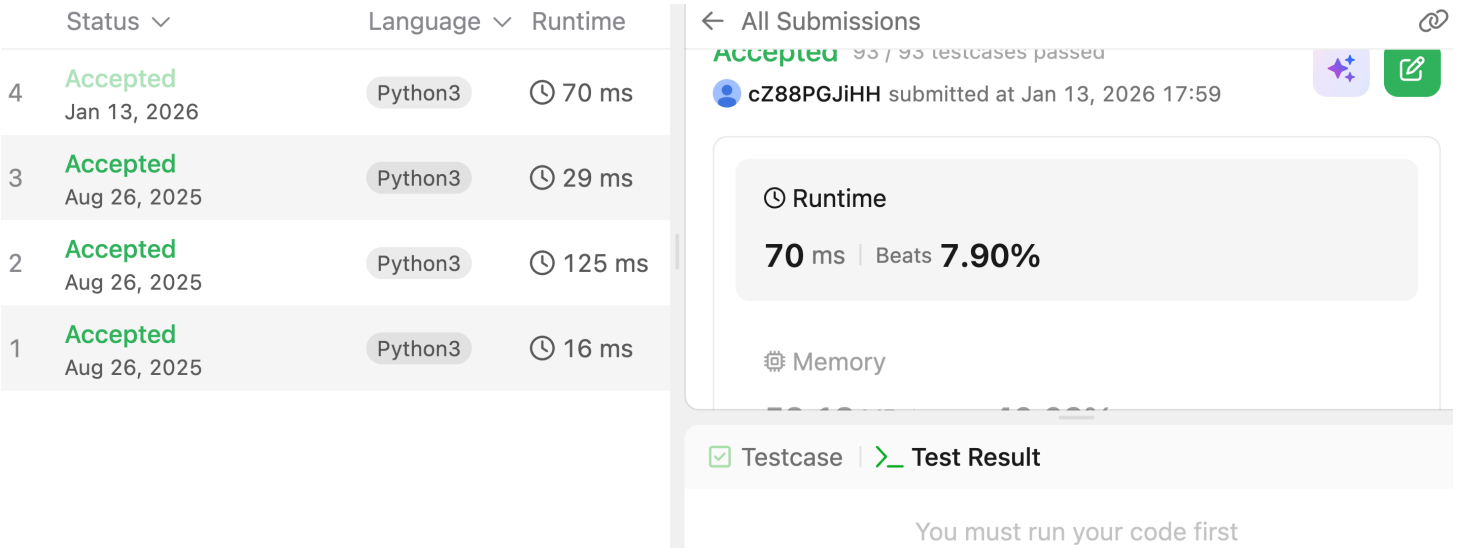
linked list, two pointers, https://leetcode.cn/problems/palindrome-linked-list/

请用快慢指针实现 `O(1)` 空间复杂度。

思路:

代码:

```python
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def isPalindrome(self, head) -> bool:
        slow = fast = head
        prev = None
        while fast and fast.next:
            prev = ListNode(slow.val,prev)
            slow = slow.next
            fast = fast.next.next
        if fast:
            slow = slow.next
        while slow:
            if prev.val != slow.val:
                return False
            prev = prev.next
            slow = slow.next
        return True
```



M24591:中序表达式转后序表达式

stack, http://cs101.openjudge.cn/practice/24591/

思路:

代码:

```python
import sys
input = sys.stdin.read()
data = input.split()
k = int(data[0])
i = 1
for _ in range(k):
    mystr = data[i]
    num = ""
    output = []
    flagshu = True
    stack = []
    for s in mystr:
        if s.isdigit():
            flagshu = True
            num += s
        elif s == ".":
            flagshu = True
            num += s
        elif s == "*" or s == "/":
            if flagshu:
                flagshu = False
                if num:
                    output.append(num)
                num = ''
            while stack:
                a = stack.pop()
                if a == "(" or a == "+" or a == "-":
                    stack.append(a)
                    stack.append(s)
                    break
                else:
                    output.append(a)
            if not stack:
                stack.append(s)
        elif s == "(":
            if flagshu:
                flagshu = False
                if num:
                    output.append(num)
                num = ''
            stack.append(s)
        elif s == ")":
            if flagshu:
```

```
            flagshu = False
            if num:
                output.append(num)
            num = ''
        while True:
            a = stack.pop()
            if a == "(":
                break
            output.append(a)
    elif s == "+" or s == "-":
        if flagshu:
            flagshu = False
            if num:
                output.append(num)
            num = ''
        while stack:
            a = stack.pop()
            if a == "(":
                stack.append(a)
                stack.append(s)
                break
            else:
                output.append(a)
        if not stack:
            stack.append(s)
if num:
    output.append(num)
while stack:
    output.append(stack.pop())
print(*output)
i += 1
```

状态: Accepted

Source Code

```
import sys
input = sys.stdin.read()
data = input.split()
k = int(data[0])
i = 1
for _ in range(k):
    mystr = data[i]
    num = ""
    output = []
    flagshu = True
    stack = []
    for s in mystr:
        if s.isdigit():
            flagshu = True
            num += s
        elif s == ".":
            flagshu = True
            num += s
        elif s == "*" or s == "/":
            if flagshu:
                flagshu = False
```

基本信息

#: 51681873
题目: 24591
提交人: 25n2500010774(sleepy(25n2500010774)
内存: 7536kB
时间: 30ms
语言: Python3
提交时间: 2026-01-17 18:48:48

M146.LRU缓存

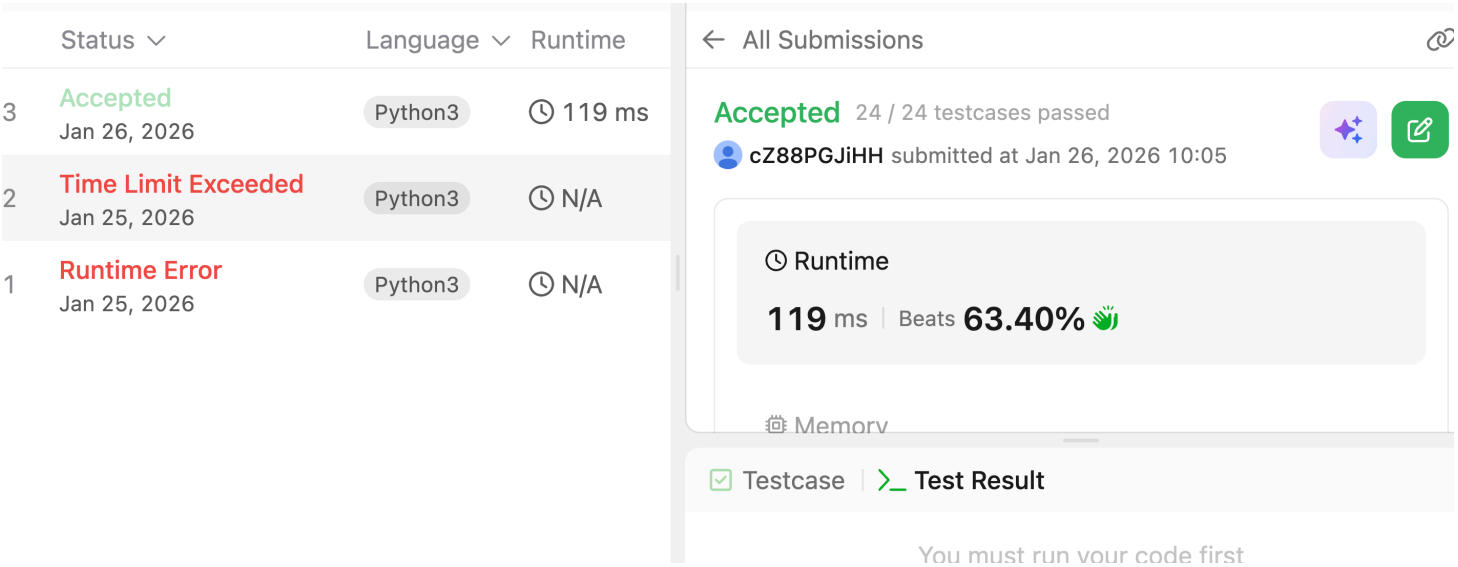
hash table, doubly-linked list, https://leetcode.cn/problems/lru-cache/

思路:

代码

```python
class ListNode:
    def __init__(self,val=-1,prev=None,next=None):
        self.val = val
        self.prev = prev
        self.next = next
class LRUCache:
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.mydict = {}
        self.mydict1 = {}
        self.node = ListNode()
        self.diu = self.node
    def get(self, key: int) -> int:
        if key in self.mydict:
            value = self.mydict[key].val
            node = ListNode(value)
            self.node.next = node
            node.prev = self.node
            self.node = node
            cur = self.mydict[key]
            self.mydict[key] = node
            del self.mydict1[cur]
            self.mydict1[node] = key
            if cur != self.diu:
                cur.prev.next = cur.next
                cur.next.prev = cur.prev
            else:
                self.diu = self.diu.next
            return value
        else:
            return -1
    def put(self, key: int, value: int) -> None:
        if key in self.mydict:
            node = ListNode(value)
            self.node.next = node
            node.prev = self.node
            self.node = node
            cur = self.mydict[key]
            self.mydict[key] = node
            del self.mydict1[cur]
            self.mydict1[node] = key
            if cur != self.diu:
                cur.prev.next = cur.next
```

```
            cur.next.prev = cur.prev
        else:
            self.diu = self.diu.next
    else:
        if len(self.mydict) == self.capacity:
            key1 = self.mydict1[self.diu]
            del self.mydict[key1]
            del self.mydict1[self.diu]
            node = ListNode(value)
            self.node.next = node
            node.prev = self.node
            self.node = node
            self.mydict[key] = node
            self.mydict1[node] = key
            self.diu = self.diu.next
        else:
            node = ListNode(value)
            self.node.next = node
            node.prev = self.node
            self.node = node
            self.mydict[key] = node
            self.mydict1[node] = key
            if self.diu.val == -1:
                self.diu = self.diu.next
```



P2698 [USACO12MAR] Flowerpot S

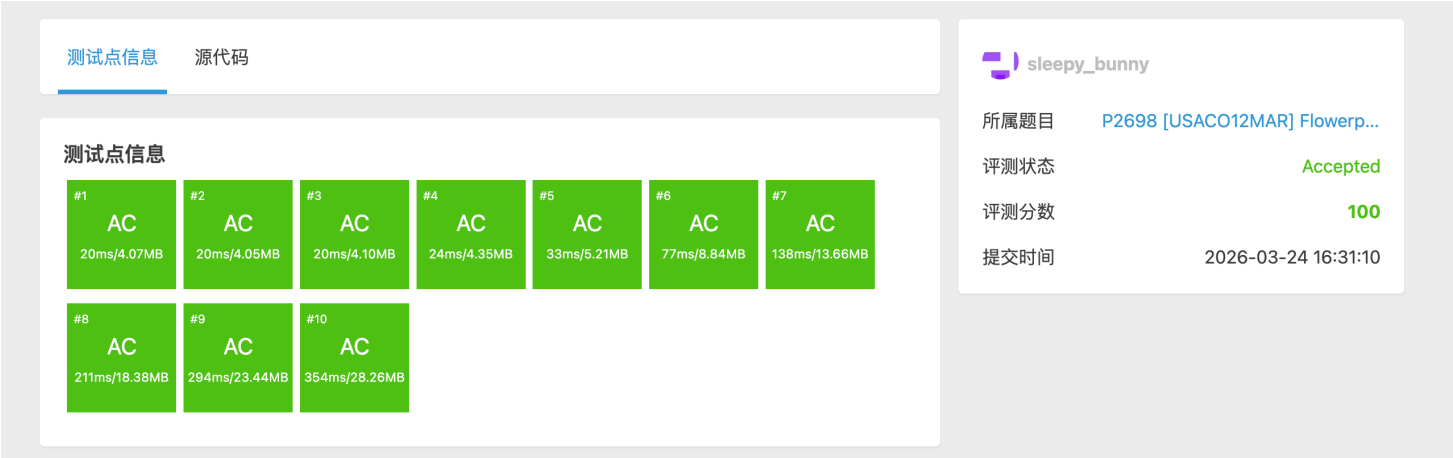
monotonic queue, https://www.luogu.com.cn/problem/P2698

思路：

代码

```python
import sys
from collections import deque
data = iter(sys.stdin.read().split())
N = int(next(data))
D = int(next(data))
yudi = []
for _ in range(N):
    x = int(next(data))
    y = int(next(data))
    yudi.append((x,y))
yudi.sort()
upper = deque([yudi[0][1]])
lower = deque([yudi[0][1]])
b,e,d= 0,0,0
length = float("inf")
flag = True
while flag:
    flag = False
    while d < D:
        if e == N-1:
            break
        flag = True
        e += 1
        while upper and upper[-1] < yudi[e][1]:
            upper.pop()
        upper.append(yudi[e][1])
        while lower and lower[-1] > yudi[e][1]:
            lower.pop()
        lower.append(yudi[e][1])
        d = upper[0]-lower[0]
    if d >= D:
        length = min(length,yudi[e][0]-yudi[b][0])
    while d >= D and b < e:
        flag = True
        b += 1
        if upper and upper[0] == yudi[b-1][1]:
            upper.popleft()
        if lower and lower[0] == yudi[b-1][1]:
            lower.popleft()
        d = upper[0]-lower[0]
        if d < D:
            break
        else:
```

```
            length = min(length,yudi[e][0]-yudi[b][0])
if length != float("inf"):
    print(length)
else:
    print(-1)
```



2. 学习总结和个人收获

每日选做，讲义的题